

任务 2 · 概念题笔记（超参数 / 优化器 / Dataset 与 DataLoader）

每题三段：原理 → Python 语法落地（建模时怎么写）→ 挂回本项目。

1. 什么是超参数？如何设定？

原理

- **参数 (parameter)**：模型在训练中**自己学**出来的数 —— 卷积核里的权重、全连接的权重矩阵、偏置。梯度下降就是不断更新它们。
- **超参数 (hyperparameter)**：训练开始**之前由人定**的"关于训练的设置"，模型自己不会去学它。它控制的是"学习这件事本身"。

类别	例子	在本项目里对应
训练过程类	学习率 lr、batch_size、epoch 数	config.py 里的 <code>LEARNING_RATE / BATCH_SIZE / EPOCHS</code>
结构类	卷积层数、通道数、卷积核大小、dropout 的 p	model.py 里 conv1~conv5、fc 的设计
正则化类	weight_decay、数据增强强度	transforms.py 里增强算子的选择

怎么设定（没有公式，靠"起点 + 对比实验"）

1. **经验起点**：Adam lr=1e-4 或 1e-3 起步；batch_size 32/64；epoch 先小（能跑通）再加大。
2. **控制变量对比**：一次只改一个超参数，其余不动，看验证集表现 —— Titanic 里你做过 Adam vs SGD 的对照，就是这个方法。
3. **看曲线调**：loss 不降 → lr 太小或不收敛；loss 震荡爆炸 → lr 太大；train acc 涨、val acc 掉 → 过拟合（加增强/dropout，或减 epoch）。
4. **用验证集选，绝不用测试集调参** —— 测试集是"最终考卷"，拿它调参等于考前偷看考卷，成绩虚高。

```
# 超参数就是普通变量，集中放 config，实验时改这里即可
LEARNING_RATE = 1e-4
BATCH_SIZE = 32
EPOCHS = 15

optimizer = torch.optim.Adam(model.parameters(), lr=LEARNING_RATE)
loader = torch.utils.data.DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True)
for epoch in range(EPOCHS):    # epoch 数也是超参数
    ...
```

2. 什么是优化器？有哪些类型？

原理

优化器 = 决定“怎么用梯度更新参数”的算法。反向传播只算出梯度（往哪个方向、多陡），优化器决定**步子怎么迈**：走多快、有没有惯性、要不要给每个参数自适应步长。

优化器	一句话原理	特点
SGD	沿梯度反方向走固定步长	朴素；lr 不好调，容易在沟谷里震荡
SGD + Momentum	累积历史更新方向，像有惯性的小球	冲过局部震荡，收敛更快更稳
AdaGrad / RMSProp	给每个参数自适应步长（常更新的参数步子变小）	适合稀疏特征；RMSProp 修复 AdaGrad 后劲不足
Adam	Momentum（惯性）+ RMSProp（自适应步长）合体	起点首选，对 lr 不敏感；AdamW = 修复了 weight_decay 写法的 Adam

怎么选：不确定就用 Adam(lr=1e-4) 起步；追求更好泛化再试 SGD+momentum（大模型时代 CNN 常用），但要配好 lr。你 Titanic 的实测（Adam 0.8156 vs SGD 0.6369）就是“小数据 + 特征尺度差异大”场景下 Adam 占优的直接证据——图像任务上值得再对比一次，结论可能不同。

```
optimizer = torch.optim.Adam(model.parameters(), lr=1e-4)
optimizer = torch.optim.SGD(model.parameters(), lr=0.01, momentum=0.9)
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-4, weight_decay=1e-4)
```

训练循环里的标准四步（优化器只参与 ① 和 ④）：

```
optimizer.zero_grad()    # ① 清梯度（不清会累加！）
loss.backward()          # ② 反向传播算梯度
optimizer.step()         # ③ 按优化器规则更新参数
```

3. DataLoader 与 DataSet 是什么及其使用？

原理

- **Dataset（数据集抽象）**：一个“仓库”的清单规则。PyTorch 要求它实现两件事：
 - `__len__()` —— 一共有多少条数据；
 - `__getitem__(i)` —— 按编号取第 *i* 条（返回（图片张量，标签））。有了这两条，才能“按编号取货”。`ImageFolder` 就是官方写好的现成 Dataset：文件夹名当标签，自动编号。
- **DataLoader（数据装载机）**：在 Dataset 外面套的“卡车队”，负责把零散的数据组装成批次喂给模型：
 - `batch_size` —— 每车装几张（GPU 一次算一批，比一张张算快得多）；
 - `shuffle` —— 每 epoch 打乱顺序（训练集必须打乱，否则模型按文件夹顺序“背答案”）；
 - `num_workers` —— 几个后台进程同时读图解码（读图是 CPU 活，别让 GPU 等饭吃）；
 - `collate_fn` —— 怎么把若干张图拼成一个 batch 张量（默认行为就够用）。

关系一句话：**DataLoader 里装着 Dataset；Dataset 只管“怎么取一条”，DataLoader 管“怎么高效成批地喂”。**

Python 语法落地

```
from torch.utils.data import Dataset, DataLoader
from torchvision import datasets
from PIL import Image

# — 用法一（本项目用的）：现成的 ImageFolder，目录结构即标签 —
# train/
# |— daisy/xxx.jpg      -> 自动标签 0
# |— rose/yyy.jpg       -> 自动标签 1
ds = datasets.ImageFolder("data/processed/train", transform=train_transform)
print(ds.class_to_idx)  # {'daisy': 0, 'dandelion': 1, 'rose': 2, 'sunflower': 3, 'tulip': 4}

# — 用法二：自定义 Dataset 的最小模板（读懂即可，本题不需要写） —
class MyDataset(Dataset):
    def __init__(self, file_list, labels, transform):
        self.files = file_list
        self.labels = labels
        self.transform = transform
    def __len__(self):
        return len(self.files)
    def __getitem__(self, idx):
        img = Image.open(self.files[idx]).convert("RGB")
        return self.transform(img), self.labels[idx]

# — DataLoader 套在外面 —
loader = DataLoader(ds, batch_size=32, shuffle=True, num_workers=2)
for images, labels in loader:      # images: [32, 3, 224, 224]  labels: [32]
    ...
```

挂回本项目

- `src/data/dataset.py` 的 `make_loaders()`：三个 `ImageFolder` (train 用增强版 transform, val/test 用无随机版) + 三个 `DataLoader`。
- train/val 来自 `split.py` 划分的 `processed/`；test 直接指向原始目录（坏图已被 `clean.py` 隔离）。